



Polynomial-time approximation algorithms for weighted LCS problem[☆]

M. Cygan, M. Kubica^{*}, J. Radoszewski, W. Rytter, T. Walen[†]

Faculty of Mathematics, Informatics and Mechanics, University of Warsaw, ul. Banacha 2, 02-097 Warsaw, Poland

ARTICLE INFO

Article history:

Received 27 November 2012

Received in revised form 8 November 2015

Accepted 12 November 2015

Available online 7 December 2015

Keywords:

Longest common subsequence

Weighted sequence

Polynomial-time approximation scheme

ABSTRACT

We consider a variant of the well-known Longest Common Subsequence (LCS) problem for weighted sequences. A weighted sequence determines the probability for each symbol to occur at a given position of the sequence (such sequences are also called Position Weighted Matrices, PWM). Two possible such versions of the problem were proposed by (Amir et al., 2009 and 2010), they are called LCWS and LCWS2 (Longest Common Weighted Subsequence 1 and 2). We solve an open problem, stated in the paper by Amir et al., of the tractability of a log-probability version of LCWS2 problem for bounded alphabets, showing that it is NP-hard already for an alphabet of size 2. We also improve the $(1/|\Sigma|)$ -approximation algorithm given by Amir et al. (where Σ is the alphabet): we show a polynomial-time approximation scheme (PTAS) for the LCWS2 problem using $O(n^5)$ space. We also give a simpler $(1/2)$ -approximation algorithm for the same problem using only $O(n^2)$ space.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

We consider (biological) *weighted sequences*, in which for each position we know the probability of an occurrence of any symbol from the alphabet Σ (more formal definition follows). Weighted sequences are also referred to in the literature as p-weighted sequences or Position Weighted Matrices (PWM) [2,16]. The notion of weighted sequence was introduced as a tool for motif discovery and local alignment, and is extensively used in computational molecular biology. In particular, binding sites, profiles of protein families and complete chromosome sequences that have been obtained using a whole-genome shotgun strategy [15,17] can be modelled as weighted sequences [10].

Multiple algorithmic results related to combinatorics of weighted sequences, i.e., repetitions, regularities and pattern matching, have already been presented. The basic concepts (including pattern matching, repeats discovery and cover computation) were studied using weighted suffix trees [11], Crochemore partition [5,13], Karp–Miller–Rabin algorithm [7], and other approaches [22]. There are also results dealing with: approximate and gapped pattern matching [3,20], property matching [1], swapped matching [19], all-covers and all-seeds problem [18,21], and extracting motifs (repeated motifs, common motifs and all maximal pairs) from weighted sequences [14]. On the practical side, there are recent results concerning massive exact and approximate pattern matching for mapping short weighted sequences to a reference genome [4], also in the parallel setting [12].

[☆] A preliminary version of this work appeared in [8].

^{*} Corresponding author. Tel.: +48 225 544 433; fax: +48 225 544 400.

E-mail addresses: cygan@mimuw.edu.pl (M. Cygan), kubica@mimuw.edu.pl (M. Kubica), jrad@mimuw.edu.pl (J. Radoszewski), rytter@mimuw.edu.pl (W. Rytter), walen@mimuw.edu.pl (T. Walen[†]).

<http://dx.doi.org/10.1016/j.dam.2015.11.011>

0166-218X/© 2015 Elsevier B.V. All rights reserved.

x_1	x_2	x_3	x_4
$p_1(\mathbf{a}) = 1/3$	$p_2(\mathbf{a}) = 1$	$p_3(\mathbf{a}) = 0$	$p_4(\mathbf{a}) = 1/2$
$p_1(\mathbf{b}) = 1/3$	$p_2(\mathbf{b}) = 0$	$p_3(\mathbf{b}) = 1/2$	$p_4(\mathbf{b}) = 1/4$
$p_1(\mathbf{c}) = 1/3$	$p_2(\mathbf{c}) = 0$	$p_3(\mathbf{c}) = 1/2$	$p_4(\mathbf{c}) = 1/4$

Fig. 1. A weighted sequence $X = x_1x_2x_3x_4$ over the alphabet $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$.

Recently Amir et al. [2] extended another well-known string problem, the Longest Common Subsequence problem [6], to weighted sequences. They introduced two versions of the Longest Common Weighted Subsequence problem, LCWS and LCWS2. Despite their similarity the complexity status of these problems is dramatically different. The former has a polynomial time solution, while the latter is NP-hard. There are other papers [1,3] dealing with the LCS problem for weighted sequences, however none of them considers the LCS problem explicitly (the first one defines and considers weighted Hamming and edit distances, while the second considers a general property matching setting). Moreover, both papers are limited by the assumption that patterns are ordinary strings.

The main problem considered here is LCWS2. We solve an open problem stated by Amir et al. [2] and show that a log-probability version of LCWS2 is NP-hard for a bounded alphabet, moreover, even for alphabet of size 2—the proof can be found in Section 3. We also improve the $(1/|\Sigma|)$ -approximation algorithm for LCWS2 proposed in [2] by providing a polynomial-time approximation scheme (PTAS) for LCWS2. Additionally we give a simpler $(1/2)$ -approximation algorithm with smaller space requirements—it uses $O(n^2)$ space instead of $O(n^5)$ space required by the PTAS. Both algorithms are described in Section 4. The paper ends with a Conclusions section. In the following section we recall the definitions of the respective problems, for which we propose a novel, more abstract formulation.

2. Preliminaries

Let Σ be a finite alphabet, $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_K\}$. We will be assuming for simplicity, as in most applications, that $K = |\Sigma| = O(1)$. By Σ^* we denote the set of all strings over Σ . By Σ^d we denote the set of strings of length d .

Definition 2.1 (Weighted Sequence). A weighted sequence $X = x_1x_2 \dots x_m$ of length $|X| = m$ over an alphabet $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_K\}$ is a sequence of sets of pairs of the form:

$$x_i = \{(\sigma_j, p_i^{(X)}(\sigma_j)) : j = 1, 2, \dots, K\}.$$

If the considered weighted sequence is unambiguous, we will write p_i instead of $p_i^{(X)}$. Here $p_i(\sigma_j)$ is the occurrence probability of the character σ_j at the position $i \in \{1, \dots, m\}$, these values are non-negative and sum up to 1 for a given i .

By $\mathcal{WS}(\Sigma)$ we denote the set of all weighted sequences over the alphabet Σ .

An example of a weighted sequence of length 4 can be found in Fig. 1.

Now we recall the definitions of two versions of the Longest Common Weighted Subsequence problem [2]. We simplify their formulation by introducing an auxiliary notion of α -subsequence.

Let $X \in \mathcal{WS}(\Sigma)$. Let $\text{Seq}_d^{|X|}$ be the set of all increasing sequences of d positions in X . For a string $s \in \Sigma^d$ and $\pi \in \text{Seq}_d^{|X|}$, define $\mathcal{P}_X(\pi, s)$ as the probability that the substring on positions corresponding to π in X equals s . More formally, if $\pi = (i_1, i_2, \dots, i_d)$ then

$$\mathcal{P}_X(\pi, s) = \prod_{k=1}^d p_{i_k}^{(X)}(s_k). \quad (1)$$

$$\text{Denote } \text{SUBS}(X, \alpha) = \left\{ s \in \Sigma^* : \exists \left(\pi \in \text{Seq}_{|s|}^{|X|} \right) \mathcal{P}_X(\pi, s) \geq \alpha \right\}. \quad (2)$$

In other words $\text{SUBS}(X, \alpha)$ is the set of strings which match some subsequence of X with probability at least α . Every $s \in \text{SUBS}(X, \alpha)$ is called an α -subsequence of X . An α -subsequence of length d is also called an (α, d) -subsequence.

Example 2.2. Consider the weighted sequence X from Fig. 1. The string $\mathbf{a}$ is its α -subsequence for any $\alpha \in (0, 1]$, the string $\mathbf{aba}$ is its $\frac{1}{4}$ -subsequence, while the string $\mathbf{aaaa}$ is not its α -subsequence for any $\alpha > 0$. Moreover, for this sequence X we have:

$$\text{SUBS}(X, \frac{1}{4}) = \{\mathbf{a}, \mathbf{aa}, \mathbf{ab}, \mathbf{aba}, \mathbf{ac}, \mathbf{aca}, \mathbf{b}, \mathbf{ba}, \mathbf{c}, \mathbf{ca}\}.$$

Definition 2.3 (α -LCWS Problem).

Input: Two weighted sequences $X, Y \in \mathcal{WS}(\Sigma)$ and a cut-off probability α .

Output: The longest string s for which there exist probabilities α_1, α_2 such that $\alpha_1\alpha_2 \geq \alpha$ and $s \in \text{SUBS}(X, \alpha_1) \cap \text{SUBS}(Y, \alpha_2)$.

Example 2.4. Consider the following weighted sequences X, Y :

X	1	2	3	4	5	Y	1	2	3	4	5
a	0.9	0.5	0.1	0.2	0.8	a	0.9	0.2	1.0	0.3	0.9
b	0.1	0.5	0.9	0.8	0.2	b	0.1	0.8	0.0	0.7	0.1

The string $s = abba$ is the solution of α -LCWS problem for $\alpha = 0.23$.

$$\mathcal{P}_X(\pi' = (1, 3, 4, 5), s) = 0.9 \cdot 0.9 \cdot 0.8 \cdot 0.8 = 0.5184$$

$$\mathcal{P}_Y(\pi = (1, 2, 4, 5), s) = 0.9 \cdot 0.8 \cdot 0.7 \cdot 0.9 = 0.4536$$

$$\mathcal{P}_X(\pi, s) \cdot \mathcal{P}_Y(\pi', s) = 0.23514624.$$

Definition 2.5 ((α_1, α_2) -LCWS2 Problem).

Input: Two weighted sequences X, Y and two cut-off probabilities α_1, α_2 .

Output: The longest string $s \in SUBS(X, \alpha_1) \cap SUBS(Y, \alpha_2)$.

Example 2.6. Consider the following weighted sequences X, Y :

X	1	2	3	4	5	Y	1	2	3	4	5
a	0.9	0.5	0.1	0.2	0.8	a	0.9	0.2	1.0	0.3	0.9
b	0.1	0.5	0.9	0.8	0.2	b	0.1	0.8	0.0	0.7	0.1

The string $s = aba$ is the solution of (α_1, α_2) -LCWS2 problem with $\alpha_1 = 0.6, \alpha_2 = 0.7$.

$$\mathcal{P}_X(\pi = (1, 3, 5), s) = 0.9 \cdot 0.9 \cdot 0.8 = 0.648$$

$$\mathcal{P}_Y(\pi' = (1, 2, 3), s) = 0.9 \cdot 0.8 \cdot 1.0 = 0.72.$$

In both problems by n we denote $\max(|X|, |Y|)$. The complexity status of these problems is dramatically different, the α -LCWS problem has a polynomial time solution, while a log-probability version of the (α_1, α_2) -LCWS2 problem is NP-hard (formal definition of this version can be found in Section 3).

Theorem 2.7 ([2]). (a) The α -LCWS problem can be solved in $O(n^3)$ time and $O(n^2)$ space. If we are only interested in the length of the solution, the problem can be solved in $O(Ln^2)$ time, where L is the length of the solution.

(b) The log-probability version of the (α_1, α_2) -LCWS2 problem is NP-hard over unbounded alphabets and admits a $(1/|\Sigma|)$ -approximation algorithm (thus the problem itself admits the same approximation algorithm).

The main problem considered in this paper is the LCWS2 problem (however, we also use as a tool the polynomial time solution of the LCWS problem). We tackle its version with a single cut-off probability, stated in Definition 2.8, which we prove below to be equivalent to the general version with two probabilities.

Definition 2.8 (α -LCWS2 Problem).

Input: Two weighted sequences $X, Y \in \mathcal{WS}(\Sigma)$ and a cut-off probability α .

Output: The longest string $s \in SUBS(X, \alpha) \cap SUBS(Y, \alpha)$.

In Lemma 2.9 we show a reduction of (α_1, α_2) -LCWS2 problem to α -LCWS2 problem. More precisely, we show that any instance (X, Y) of (α_1, α_2) -LCWS2 problem can be transformed into an instance (X', Y') of $\min(\alpha_1, \alpha_2)$ -LCWS2 problem having the same solution. The transformation used in the lemma is illustrated in Example 2.10. Intuitively, if $\alpha_1 < \alpha_2 < 1$, X' is chosen as X and probabilities in Y' are the probabilities from Y taken to $\log_{\alpha_2} \alpha_1$ power, so that an α_2 -subsequence of Y is an α_1 -subsequence ($\alpha_2^{\log_{\alpha_2} \alpha_1} = \alpha_1$) of Y' . We introduce the symbol $\#$ in the alphabet for X' and Y' so that the probabilities sum up to 1 on each position.

Lemma 2.9. Let $X, Y \in \mathcal{WS}(\Sigma)$ and let $\alpha_1, \alpha_2 \in (0, 1], \alpha_1 < \alpha_2$. There exist $X', Y' \in \mathcal{WS}(\Sigma')$ ($|X'| = |X|, |Y'| = |Y|$), where $\Sigma' = \Sigma \cup \{\#\}$ is the original alphabet extended by a symbol $\# \notin \Sigma$, such that for any string s :

s is a solution to the (α_1, α_2) -LCWS2 problem for X and $Y \Leftrightarrow$

s is a solution to the α_1 -LCWS2 problem for X' and Y' .

Moreover, weighted sequences X', Y' can be constructed from X, Y in $O(n)$ time.

Proof. First assume that $\alpha_2 < 1$. Let $\gamma = \log_{\alpha_2} \alpha_1$. Recall that $\Sigma = \{\sigma_1, \dots, \sigma_K\}$. We define weighted sequences X' and Y' over $\Sigma' = \Sigma \cup \{\#\}$ by the following probabilities:

$$\begin{aligned} p_i^{(X')}(\sigma_j) &= p_i^{(X)}(\sigma_j), \quad \text{for } i \in \{1, \dots, |X|\}, j \in \{1, \dots, K\} \\ p_i^{(X')}(\#) &= 0, \quad \text{for } i \in \{1, \dots, |X|\} \\ p_i^{(Y')}(\sigma_j) &= p_i^{(Y)}(\sigma_j)^\gamma, \quad \text{for } i \in \{1, \dots, |Y|\}, j \in \{1, \dots, K\} \\ p_i^{(Y')}(\#) &= 1 - \sum_{j=1}^k p_i^{(Y)}(\sigma_j) \quad \text{for } i \in \{1, \dots, |Y|\}. \end{aligned}$$

Let us note that all the probabilities $p_i^{(X')}(\sigma_j), p_i^{(Y')}(\sigma_j)$ are non-negative, do not exceed 1 (this is because $\gamma > 1$) and sum up to 1 for a given position i in the weighted sequence X' and Y' , respectively.

We show that all solutions of α_1 -LCWS2 problem for X' and Y' are solutions of (α_1, α_2) -LCWS2 for X and Y , and conversely. For this it suffices to argue that (see [Definitions 2.5](#) and [2.8](#)):

$$\text{SUBS}(X, \alpha_1) \cap \text{SUBS}(Y, \alpha_2) = \text{SUBS}(X', \alpha_1) \cap \text{SUBS}(Y', \alpha_1). \quad (3)$$

Note that the left side of the equality (3) is a subset of Σ^* , while the right side is a subset of $(\Sigma')^*$. We prove (3) by showing two inclusions.

($\subseteq$) Let $s \in \text{SUBS}(X, \alpha_1) \cap \text{SUBS}(Y, \alpha_2)$, $s = s_1 s_2 \dots s_d$. Then obviously $s \in \text{SUBS}(X', \alpha_1)$. Let $\pi \in \text{Seq}_d^{|Y|}$, $\pi = (i_1, i_2, \dots, i_d)$, be a sequence of positions for which $\mathcal{P}_Y(\pi, s) \geq \alpha_2$. Then the same sequence of positions shows that $s \in \text{SUBS}(Y', \alpha_1)$:

$$\mathcal{P}_{Y'}(\pi, s) = \prod_{j=1}^d p_{i_j}^{(Y')}(\sigma_j) = \prod_{j=1}^d p_{i_j}^{(Y)}(\sigma_j)^\gamma = \mathcal{P}_Y(\pi, s)^\gamma \geq \alpha_2^\gamma = \alpha_1.$$

($\supseteq$) Let $s \in \text{SUBS}(X', \alpha_1) \cap \text{SUBS}(Y', \alpha_1)$, $s = s_1 s_2 \dots s_d$. First note that $s \in \Sigma^*$, otherwise s would not be an α_1 -subsequence of X' (since $p_i^{(X')}(\#) = 0$ for all $i \in \{1, \dots, |X'|\}$). Hence, $s \in \text{SUBS}(X, \alpha_1)$. Let $\pi \in \text{Seq}_d^{|Y'|}$, $\pi = (i_1, i_2, \dots, i_d)$, be a sequence of positions for which $\mathcal{P}_{Y'}(\pi, s) \geq \alpha_1$. Then the same sequence of positions shows that $s \in \text{SUBS}(Y, \alpha_2)$:

$$\mathcal{P}_Y(\pi, s) = \prod_{j=1}^d p_{i_j}^{(Y)}(\sigma_j) = \prod_{j=1}^d p_{i_j}^{(Y')}(\sigma_j)^{1/\gamma} = \mathcal{P}_{Y'}(\pi, s)^{1/\gamma} \geq \alpha_1^{1/\gamma} = \alpha_2.$$

We are left with the boundary case $\alpha_2 = 1$. In this case we define the weighted sequence X' as before, and Y' as:

$$\begin{aligned} p_i^{(Y')}(\sigma_j) &= 1 \quad \text{if } p_i^{(Y)}(\sigma_j) = 1 \quad \text{and} \quad p_i^{(Y')}(\sigma_j) = 0 \quad \text{otherwise (for } i \in \{1, \dots, |Y|\}, j \in \{1, \dots, K\}), \\ p_i^{(Y')}(\#) &= 1 - \sum_{j=1}^k p_i^{(Y)}(\sigma_j) \quad \text{for } i \in \{1, \dots, |Y|\}. \end{aligned}$$

As previously, we show the following equality by two inclusions:

$$\text{SUBS}(X, \alpha_1) \cap \text{SUBS}(Y, 1) = \text{SUBS}(X', \alpha_1) \cap \text{SUBS}(Y', \alpha_1). \quad (4)$$

($\subseteq$) Let $s \in \Sigma^*$. If $s \in \text{SUBS}(X, \alpha_1)$ then obviously $s \in \text{SUBS}(X', \alpha_1)$. Moreover, if $s \in \text{SUBS}(Y, 1)$ then $s \in \text{SUBS}(Y', 1)$, therefore $s \in \text{SUBS}(Y', \alpha_1)$. This concludes the proof of this inclusion.

($\supseteq$) Let $s \in \text{SUBS}(X', \alpha_1) \cap \text{SUBS}(Y', \alpha_1)$. Note that, as previously, s has to be an element of Σ^* . Hence, $s \in \text{SUBS}(X, \alpha_1)$. Further note that any α_1 -subsequence of Y' is actually a 1-subsequence of Y' . This implies that $s \in \text{SUBS}(Y', 1)$ and consequently $s \in \text{SUBS}(Y, 1)$. This concludes the proof of the lemma. $\square$

Example 2.10. Consider the weighted sequences X, Y from [Example 2.6](#) and $\alpha_1 = 0.6, \alpha_2 = 0.7$.

X	1	2	3	4	5	Y	1	2	3	4	5
a	0.9	0.5	0.1	0.2	0.8	a	0.9	0.2	1.0	0.3	0.9
b	0.1	0.5	0.9	0.8	0.2	b	0.1	0.8	0.0	0.7	0.1

By Lemma 2.9, in order to solve $(0.6, 0.7)$ -LCWS2 problem for X and Y it suffices to solve 0.6-LCWS2 problem for the following two weighted sequences X', Y' . Here $\gamma = \log_{\alpha_2} \alpha_1 \approx 1.4322$.

X'	1	2	3	4	5	Y'	1	2	3	4	5
a	0.9	0.5	0.1	0.2	0.8	a	0.860	0.100	1.000	0.178	0.860
b	0.1	0.5	0.9	0.8	0.2	b	0.037	0.726	0.000	0.600	0.037
#	0.0	0.0	0.0	0.0	0.0	#	0.103	0.174	0.000	0.222	0.103

For the string $s = aba$ we have:

$$\mathcal{P}_X(\pi = (1, 3, 5), s) = 0.9 \cdot 0.9 \cdot 0.8 = 0.648$$

$$\mathcal{P}_Y(\pi' = (1, 2, 3), s) = 0.9 \cdot 0.8 \cdot 1.0 = 0.72$$

$$\mathcal{P}_{X'}(\pi, s) = 0.9 \cdot 0.9 \cdot 0.8 = 0.648$$

$$\mathcal{P}_{Y'}(\pi', s) \approx 0.860 \cdot 0.726 \cdot 1.0 \approx 0.624.$$

3. Integer LCWS2 over a bounded alphabet is NP-hard

The integer log-probability version of the LCWS2 problem, as defined by Amir et al. [2], is formulated in terms of l -weighted sequences. An l -weighted sequence $X = x_1 x_2 \dots x_m$ over an alphabet $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_K\}$ is a sequence of sets of pairs of the form:

$$x_i = \{(\sigma_j, w_i^{(X)}(\sigma_j)) : j = 1, 2, \dots, K\}, \quad \text{where } w_i^{(X)}(\sigma_j) \in \mathbb{Z}_+.$$

Recall that in the (α_1, α_2) -LCWS2 problem we are given two weighted sequences X, Y and two cut-off probabilities α_1, α_2 and we are to find the longest string $s \in SUBS(X, \alpha_1) \cap SUBS(Y, \alpha_2)$. In the integer LCWS2 problem we also use the $SUBS$ -notation, which is defined for an l -weighted sequence X and $\alpha \in \mathbb{Z}_+$ as:

$$SUBS(X, \alpha) = \left\{ s \in \Sigma^* : \exists \left(\pi \in \text{Seq}_{|s|}^{[X]} \right) \mathcal{W}_X(\pi, s) \leq \alpha \right\},$$

where

$$\mathcal{W}_X(\pi, s) = \sum_{k=1}^d w_{i_k}^{(X)}(s_k) \quad \text{for } \pi = (i_1, \dots, i_d) \in \text{Seq}_d^{[X]}.$$

Now the LCIWS2 problem can be stated as follows:

Definition 3.1 ((α_1, α_2) -LCIWS2 Problem).

Input: Two l -weighted sequences X, Y and two thresholds $\alpha_1, \alpha_2 \in \mathbb{Z}_+$.

Output: The longest string $s \in SUBS(X, \alpha_1) \cap SUBS(Y, \alpha_2)$.

As in the previous section we also define a variant of the LCIWS2 problem with a single cut-off value.

Definition 3.2 (α -LCIWS2 Problem).

Input: Two l -weighted sequences X, Y and a threshold $\alpha \in \mathbb{Z}_+$.

Output: The longest string $s \in SUBS(X, \alpha) \cap SUBS(Y, \alpha)$.

The previously known proof [2] of NP-hardness of the (α_1, α_2) -LCIWS2 problem depended on the assumption of an unbounded alphabet Σ . We show NP-hardness of (α_1, α_2) -LCIWS2 over the alphabet $\Sigma = \{a, b\}$. Note that it suffices to show that the α -LCIWS2 problem is NP-hard, since it is a special case of the (α_1, α_2) -LCIWS2 problem.

We perform a reduction of α -LCIWS2 to the following NP-complete problem [9] (the same NP-complete problem was used in [2]).

Definition 3.3 (Partition Problem).

Input: A finite set $S, S \subseteq \mathbb{Z}_+$.

Binary output: Is there a subset $S' \subset S$ such that $\sum S' = \sum S \setminus S'$.

We make the reduction from the Partition problem to the α -LCIWS2 problem as follows. Let $S = \{q_1, q_2, \dots, q_n\}$ be an instance of the Partition problem. We construct l-weighted sequences $X = x_1x_2 \dots x_n$ and $Y = y_1y_2 \dots y_n$ over the alphabet $\Sigma = \{a, b\}$ with the following weights of letters from Σ :

$$w_i^{(X)}(a) = q_i + c, \quad w_i^{(X)}(b) = c, \quad w_i^{(Y)}(a) = c, \quad w_i^{(Y)}(b) = q_i + c.$$

Here $c > 0$ is an arbitrary positive integer (we use it to obtain strictly positive weights in the l-weighted sequences). Finally let $\alpha = \frac{1}{2} \sum S + nc$.

Lemma 3.4. *The Partition problem for an instance S has a positive answer iff the length of the solution to α -LCIWS2 for X and Y is n .*

Proof. ($\Rightarrow$) Let $\pi = (i_1, i_2, \dots, i_k)$ be an increasing sequence of positions such that $S' = \{q_{i_1}, q_{i_2}, \dots, q_{i_k}\}$ is a solution to the Partition problem for S , let $\pi' = (i'_1, i'_2, \dots, i'_{n-k})$ be the sequence of all remaining positions in S . Then the string $s \in \Sigma^n$ such that $s_{i_j} = a$ for i_j in π and $s_{i'_j} = b$ for i'_j in π' is a solution to the α -LCIWS2 problem for X and Y . Indeed, let $id_n = (1, 2, \dots, n)$. Then:

$$\mathcal{W}_X(id_n, s) = \sum_i w_i^{(X)}(s_i) = \sum_{i_j} (q_{i_j} + c) + \sum_{i'_j} c = \sum S' + nc = \alpha,$$

$$\mathcal{W}_Y(id_n, s) = \sum_i w_i^{(Y)}(s_i) = \sum_{i_j} c + \sum_{i'_j} (q_{i'_j} + c) = \sum (S \setminus S') + nc = \alpha,$$

thus $s \in SUBS(X, \alpha) \cap SUBS(Y, \alpha)$. Hence, s is a solution to the α -LCIWS2 problem for X and Y , since it is the longest string in this set.

($\Leftarrow$) Let $s \in \Sigma^n$ be a solution to α -LCIWS2 problem for X and Y . Denote by $\pi = (i_1, i_2, \dots, i_k)$ and $\pi' = (i'_1, i'_2, \dots, i'_{n-k})$ the increasing sequences of positions within s containing letters a and b respectively. Then, for $id_n = (1, 2, \dots, n)$, the following inequalities must hold:

$$\alpha \geq \mathcal{W}_X(id_n, s) = \sum_i w_i^{(X)}(s_i) = \sum_{i_j} (q_{i_j} + c) + \sum_{i'_j} c = \sum_{i_j} q_{i_j} + nc,$$

$$\alpha \geq \mathcal{W}_Y(id_n, s) = \sum_i w_i^{(Y)}(s_i) = \sum_{i_j} c + \sum_{i'_j} (q_{i'_j} + c) = \sum_{i'_j} q_{i'_j} + nc.$$

By recalling the definition of α and reducing equal summands, we obtain

$$\sum_{i_j} q_{i_j} \leq \frac{1}{2} \sum S \quad \text{and} \quad \sum_{i'_j} q_{i'_j} \leq \frac{1}{2} \sum S. \quad (5)$$

Note that both left sides of the inequalities (5) are non-negative and sum up to $\sum S$. Hence, both inequalities (5) are equalities, and therefore the set S' defined as $S' = \{q_{i_1}, q_{i_2}, \dots, q_{i_k}\}$ is a solution to the Partition problem for the set S . $\square$

Example 3.5. Let $S = \{2, 3, 6, 9, 10\}$ be an instance of the Partition problem. We have $\sum S = 30$ and thus we are looking for a subset $S' \subset S$ such that $\sum S' = 15$. The corresponding instance of the α -LCIWS2 problem for $c = 1$ and $\alpha = 20$ consists of the following l-weighted sequences X and Y :

X	1	2	3	4	5	Y	1	2	3	4	5
a	(3)	(4)	7	10	(11)	a	(1)	(1)	1	1	(1)
b	1	1	(1)	(1)	1	b	3	4	(7)	(10)	11

A solution aabba of length 5 to the α -LCIWS2 problem for X and Y has been highlighted. It corresponds to the subset $S' = \{2, 3, 10\}$ of S which is a solution to the Partition problem for S .

Due to Lemma 3.4, we have a reduction from the Partition problem to the α -LCIWS2 problem, and even more, to a decision version of LCIWS2 (asking whether there exists a common subsequence of a given length). Recall that α -LCIWS2 problem is just a special case of (α_1, α_2) -LCIWS2 problem, that is, any instance of the α -LCIWS2 problem is actually an instance of the (α, α) -LCIWS2 problem. We conclude that the decision version of LCIWS2 problem is NP-complete, and moreover that:

Theorem 3.6. *The (α_1, α_2) -LCIWS2 problem over a binary alphabet is NP-hard.*

4. Approximating LCWS2

Previous work on the (α_1, α_2) -LCWS2 problem [2] contained a $(1/|\Sigma|)$ -approximation algorithm. In this section we present a $(1/2)$ -approximation algorithm for (α_1, α_2) -LCWS2 and then proceed to a polynomial-time approximation scheme (PTAS) for this problem. The first algorithm is more space-efficient than the general approximation scheme.

4.1. $(1/2)$ -approximation algorithm for LCWS2

We start the description of the algorithm by showing an important connection between the α -LCWS2 and α -LCWS problems (Lemma 4.1). Afterwards we show how it can be used to obtain a $(1/2)$ -approximation algorithm for the general (α_1, α_2) -LCWS2 problem.

Let $X, Y \in \mathcal{WS}(\Sigma)$, $n = \max(|X|, |Y|)$. By $\text{OPT}(X, Y, \alpha)$ we denote the length of the solution to the α -LCWS2 problem for X and Y . In case the length of the solution to α^2 -LCWS problem for X, Y is even, we obtain a $(1/2)$ -approximation of $\text{OPT}(X, Y, \alpha)$ simply by taking a half of the solution to the α^2 -LCWS problem. In the case of odd length, we need to use the solutions to the α' -LCWS problem for all prefixes of X and Y , for different values of α' . Details can be found in the proof of the following lemma.

Lemma 4.1. *We can compute a solution to the α -LCWS2 problem for $X, Y \in \mathcal{WS}(\Sigma)$ of length at least $\lfloor \text{OPT}(X, Y, \alpha)/2 \rfloor$ in $O(n^3)$ time and $O(n^2)$ space.*

Proof. Let L be the length of the solution to the α -LCWS2 problem for X and Y and let d be the length of the solution to the α^2 -LCWS problem for X and Y . We will show that $\lfloor \frac{d}{2} \rfloor \leq L \leq d$. This suffices to conclude the proof, since d can be computed in $O(n^3)$ time and $O(n^2)$ space, see Theorem 2.7.

The inequality $d \geq L$ is a consequence of the following trivial inclusion:

$$\text{SUBS}(X, \alpha) \cap \text{SUBS}(Y, \alpha) \subseteq \bigcup_{\alpha_1, \alpha_2 \geq \alpha^2} (\text{SUBS}(X, \alpha_1) \cap \text{SUBS}(Y, \alpha_2)),$$

in which the left side is the set of candidates for the solution of the α -LCWS2 problem, while the right side are candidates for the solution to the α^2 -LCWS problem, both for the weighted sequences X and Y .

As for the other inequality, $L \geq \lfloor \frac{d}{2} \rfloor$, let $\pi \in \text{Seq}_d^{|X|}$, $\pi = (i_1, \dots, i_d)$, and $\pi' \in \text{Seq}_d^{|Y|}$, $\pi' = (i'_1, \dots, i'_d)$, be the sequences of positions corresponding to the solution $s = s_1 \dots s_d$ of α^2 -LCWS. Thus s satisfies:

$$\mathcal{P}_X(\pi, s) \cdot \mathcal{P}_Y(\pi', s) = \prod_{j=1}^d p_{i_j}^{(X)}(s_j) \cdot \prod_{j=1}^d p_{i'_j}^{(Y)}(s_j) \geq \alpha^2. \quad (6)$$

Let $g = \lfloor \frac{d}{2} \rfloor$. The left side of the inequality (6) can be written as $A \cdot B \cdot C \cdot D$, where:

$$A = \prod_{j=1}^g p_{i_j}^{(X)}(s_j), \quad B = \prod_{j=1}^g p_{i'_j}^{(Y)}(s_j), \quad C = \prod_{j=g+1}^d p_{i_j}^{(X)}(s_j), \quad D = \prod_{j=g+1}^d p_{i'_j}^{(Y)}(s_j).$$

Note that at most one of the numbers A, B, C, D can be less than α . Indeed, otherwise the product of these numbers would certainly be less than α^2 , since all of them are at most 1. Hence:

$$(A \geq \alpha \wedge B \geq \alpha) \vee (C \geq \alpha \wedge D \geq \alpha).$$

Consequently, at least one of the strings $s_1 \dots s_g$ or $s_{g+1} \dots s_d$ is a solution to the α -LCWS2 problem, therefore $L \geq \lfloor \frac{d}{2} \rfloor$. $\square$

Theorem 4.2. *There exists a $(1/2)$ -approximation algorithm for the (α_1, α_2) -LCWS2 problem which runs in $O(n^5)$ time and $O(n^2)$ space.*

Proof. Let (X, Y) be an instance of the (α_1, α_2) -LCWS2 problem. Let us consider all possible pairs of indices $i = 1, \dots, |X|$ and $j = 1, \dots, |Y|$. For each such pair, we assume that the last positions of a solution to the (α_1, α_2) -LCWS2 problem in X and Y are i and j , respectively. For one of them it is true, indeed. We also consider all such characters $\sigma \in \Sigma$, that $p_i^{(X)}(\sigma) \geq \alpha_1$ and $p_j^{(Y)}(\sigma) \geq \alpha_2$. The character σ is a candidate for the last symbol in the solution to the (α_1, α_2) -LCWS2 problem. Again, for some σ it is the last symbol of the solution, indeed.

To find the rest of the solution, we approximate the (α'_1, α'_2) -LCWS2 problem for the sequences $X' = x_1 \dots x_{i-1}$ and $Y' = y_1 \dots y_{j-1}$, where:

$$\alpha'_1 = \frac{\alpha_1}{p_i^{(X)}(\sigma)}, \quad \text{and} \quad \alpha'_2 = \frac{\alpha_2}{p_j^{(Y)}(\sigma)}.$$

First, by Lemma 2.9 we compute an instance (X'', Y'') of the α -LCWS2 problem, for some α , such that the solution of the (α'_1, α'_2) -LCWS2 problem for X', Y' is the same as the solution of the α -LCWS2 problem for X'', Y'' .

We use the algorithm from Lemma 4.1 to approximate the solution of the instance of the α -LCWS2 problem. Let $L'(i, j, \sigma)$ be the length of the optimal solution, and $d'(i, j, \sigma)$ be the length of the approximation. We have $d'(i, j, \sigma) \leq L'(i, j, \sigma) \leq 2d'(i, j, \sigma) + 1$.

The overall result is the longest approximation extended by the character σ . Its length equals $d' + 1$, where:

$$d' = \max\{d'(i, j, \sigma) : 1 \leq i \leq |X|, 1 \leq j \leq |Y|, \sigma \in \Sigma\}.$$

Note that:

$$L = \max\{L'(i, j, \sigma) : 1 \leq i \leq |X|, 1 \leq j \leq |Y|, \sigma \in \Sigma\} + 1.$$

Hence, $d' \leq L - 1 \leq 2d' + 1$ and $d' + 1 \geq \frac{L}{2}$. Therefore, the obtained result is the requested $(1/2)$ -approximation.

Recall that the alphabet is of a constant size. We have $O(n^2)$ pairs of indices i and j to consider. Since each invocation of the approximation algorithm described in Lemma 4.1 runs in $O(n^3)$ time and $O(n^2)$ space, we obtain $O(n^5)$ overall time complexity and $O(n^2)$ memory complexity. $\square$

4.2. Polynomial-time approximation scheme for LCWS2

In this section we first consider the α -LCWS2 problem. At the end we use the transformation from Lemma 2.9 to apply the results obtained to the (α_1, α_2) -LCWS2 problem (see Theorem 4.7).

Let $X, Y \in \mathcal{W}_\delta(\Sigma)$, $n = \max(|X|, |Y|)$, and $\alpha \in (0, 1]$. We say that an instance (X, Y) of the α -LCWS2 problem is a (δ, T) -power if all the positive weights in the sequence X are powers of δ , where $0 < \delta < 1$ and $\delta^{T-1} \geq \alpha > \delta^T$.

Lemma 4.3. *The α -LCWS2 problem for (δ, T) -power instances can be solved in $O(n^3T)$ time and space.*

Proof. Without the loss of generality, we can assume that $m = |Y| \leq |X| = n$. We use the dynamic programming technique. Our approach is a generalisation of the standard LCS algorithm. We have $O(n^3T)$ states, each described by a tuple (a, b, ℓ, t) , where:

- a is the position in the sequence X , $1 \leq a \leq n$;
- b is the position in the sequence Y , $1 \leq b \leq m$;
- ℓ is the length of the partial solution for $x_1 \dots x_a$ and $y_1 \dots y_b$, that is, the length of a subsequence of positions in $x_1 \dots x_a$ and $y_1 \dots y_b$, $0 \leq \ell \leq m$;
- t is a δ -based logarithm of the product of $p_i(\sigma_j)$ values of the considered subsequence of $x_1 \dots x_a$. By the definition of the (δ, T) -power, we only consider integer values of t from the range $[0, T]$; values larger than T can be unified with T .

For a state (a, b, ℓ, t) we store, as $\text{val}(a, b, \ell, t)$, the largest value β , for which there exists a string $z = z_1 z_2 \dots z_\ell \in \Sigma^\ell$, that is a (δ^ℓ, ℓ) -subsequence of $x_1 \dots x_a$ and (β, ℓ) -subsequence of $y_1 \dots y_b$. If no such subsequence exists, we set $\text{val}(a, b, \ell, t) = 0$.

Intuitively, in a state (a, b, ℓ, t) , besides the two positions a and b used in the classical LCS algorithm, we control the length of the subsequence using the parameter ℓ and with the parameter t we count the number of times the value δ is multiplied in the product of probabilities used for the subsequence of the prefix of X .

When computing the value for a state (a, b, ℓ, t) , it suffices to consider three options: we can either drop the a th position in the sequence X , or drop the b th position in the sequence Y , or use one of the $|\Sigma|$ letters at the positions a and b in the sequence X and Y respectively. Formally, we have the following recursive formula:

$$\text{val}(a, b, \ell, t) = \max\left(\text{val}(a-1, b, \ell, t), \text{val}(a, b-1, \ell, t), \max_{j=1, \dots, K} \left\{ p_b^{(Y)}(\sigma_j) \cdot \text{val}(a-1, b-1, \ell-1, t - \log_\delta(p_a^{(X)}(\sigma_j))) \right\}\right).$$

We set $\text{val}(a, b, 0, t) = 1$. For $a = 0$ or $b = 0$, and $\ell > 0$ we set $\text{val}(a, b, \ell, t) = 0$. Finally, we never select j such that $p_a^{(X)}(\sigma_j) = 0$, since in this case the weight of the subsequence in X would be 0. Thus, we can compute all the values $\text{val}(a, b, \ell, t)$ in $O(n^3T)$ time.

After computing values for all the states, we look for the largest value of d for which there exists $t \in [0, T-1]$ such that $\text{val}(|X|, |Y|, d, t) \geq \alpha$. By extending the dynamic programming algorithm in a standard manner we can construct a string $s \in \Sigma^d$ and two sequences of positions $\pi \in \text{Seq}_d^{|X|}$ and $\pi' \in \text{Seq}_d^{|Y|}$, such that $\mathcal{P}_X(\pi, s), \mathcal{P}_Y(\pi', s) \geq \alpha$. $\square$

Example 4.4. Consider the following two weighted sequences as a $(\frac{1}{2}, 4)$ -power instance of the α -LCWS2 problem with $\alpha = \frac{1}{8}$.

X	1	2	3	Y	1	2
a	$\frac{1}{2}$	0	$\frac{1}{4}$	a	$\frac{1}{3}$	$\frac{2}{3}$
b	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	b	$\frac{1}{3}$	$\frac{1}{3}$
c	0	$\frac{1}{2}$	$\frac{1}{4}$	c	$\frac{1}{3}$	0

In this example we show all the non-zero values of $\text{val}(a, b, \ell, t)$ for $\ell > 0$. All such values for $\ell = 1$ are listed below.

$$\begin{aligned} \text{val}(1, 1, 1, 1) &= \frac{1}{3} & \text{val}(2, 1, 1, 1) &= \frac{1}{3} & \text{val}(1, 2, 1, 1) &= \frac{2}{3} \\ \text{val}(2, 2, 1, 1) &= \frac{2}{3} & \text{val}(3, 2, 1, 1) &= \frac{2}{3} & \text{val}(3, 2, 1, 2) &= \frac{2}{3} \end{aligned}$$

For instance, consider the computation of $\text{val}(1, 2, 1, 1)$. By the recursive formula we have:

$$\begin{aligned} \text{val}(1, 2, 1, 1) &= \max(\text{val}(0, 2, 1, 1), \text{val}(1, 1, 1, 1), p_2^{(Y)}(a) \cdot \text{val}(0, 1, 0, 1 - \log_{\frac{1}{2}} p_1^{(X)}(a)), \\ &\quad p_2^{(Y)}(b) \cdot \text{val}(0, 1, 0, 1 - \log_{\frac{1}{2}} p_1^{(X)}(b))) \\ &= \max(0, \frac{1}{3}, \frac{2}{3} \cdot 1, \frac{1}{3} \cdot 1) = \frac{2}{3}. \end{aligned}$$

Note that we did not consider the letter c, since $p_1^{(X)}(c) = 0$.

Below we list all positive dynamic programming values for $\ell = 2$.

$$\text{val}(2, 2, 2, 2) = \frac{1}{9} \quad \text{val}(3, 2, 2, 2) = \frac{1}{9} \quad \text{val}(3, 2, 2, 3) = \frac{2}{9}$$

For instance, the values for $a = 3$ are computed as follows:

$$\begin{aligned} \text{val}(3, 2, 2, 2) &= \max(\text{val}(2, 2, 2, 2), \text{val}(3, 1, 2, 2), \frac{2}{3} \cdot \text{val}(2, 1, 1, 2 - \log_{\frac{1}{2}} \frac{1}{4}), \frac{1}{3} \cdot \text{val}(2, 1, 1, 2 - \log_{\frac{1}{2}} \frac{1}{2}), \\ &\quad 0 \cdot \text{val}(2, 1, 1, 2 - \log_{\frac{1}{2}} \frac{1}{4})) \\ &= \max(\frac{1}{9}, 0, \frac{2}{3} \cdot 0, \frac{1}{3} \cdot \frac{1}{3}, 0 \cdot 0) = \frac{1}{9}. \\ \text{val}(3, 2, 2, 3) &= \max(\text{val}(2, 2, 2, 3), \text{val}(3, 1, 2, 3), \frac{2}{3} \cdot \text{val}(2, 1, 1, 3 - \log_{\frac{1}{2}} \frac{1}{4}), \frac{1}{3} \cdot \text{val}(2, 1, 1, 3 - \log_{\frac{1}{2}} \frac{1}{2}), \\ &\quad 0 \cdot \text{val}(2, 1, 1, 3 - \log_{\frac{1}{2}} \frac{1}{4})) \\ &= \max(0, 0, \frac{2}{3} \cdot \frac{1}{3}, \frac{1}{3} \cdot 0, 0 \cdot \frac{1}{3}) = \frac{2}{9}. \end{aligned}$$

The length of the optimal subsequence is 2, since $\text{val}(3, 2, 2, 3) = \frac{2}{9} \geq \frac{1}{8} = \alpha$. The corresponding string is actually aa or ba.

Lemma 4.5. For any $\epsilon > 0$ we can compute, in $O(n^4/\epsilon)$ time and space, a string which is an $\alpha^{1+\epsilon}$ -subsequence of X and an α -subsequence of Y of length at least $\text{OPT}(X, Y, \alpha)$.

Proof. We assume that $\alpha < 1$. If $\alpha = 1$ we can solve the problem in $O(n^2)$ time and space, using a reduction to the standard LCS problem.

We start by scaling and rounding probability distributions of the weighted sequence X . Let $T = \lceil \frac{n}{\epsilon} \rceil$ and $\delta = \alpha^{1/T}$. For all i, j such that $p_i^{(X)}(\sigma_j) > 0$ we define:

$$p'_i(\sigma_j) = \delta^{\lfloor \log_{\delta}(p_i^{(X)}(\sigma_j)) \rfloor}.$$

Obviously, $p_i^{(X)}(\sigma_j) = \delta^{\log_{\delta}(p_i^{(X)}(\sigma_j))}$. Therefore:

$$p'_i(\sigma_j) \geq p_i^{(X)}(\sigma_j) \geq p'_i(\sigma_j) \alpha^{\frac{1}{T}} \geq p'_i(\sigma_j) \alpha^{\frac{\epsilon}{n}}.$$

Hence, $p'_i(\sigma_j)$ is greater than $p_i^{(X)}(\sigma_j)$ at most by a factor of $\alpha^{-\epsilon/n}$. If $p_i^{(X)}(\sigma_j) = 0$, we take $p'_i(\sigma_j) = 0$.

The conclusion follows from [Lemma 4.3](#), since with the new weight p' for the sequence X the instance (X, Y, α) is a (δ, T) -power and multiplication of at most n weights differs from α by at most a factor of α^{ϵ} . Note that the new weight p'_i is not a probability distribution, as it does not satisfy the equality $\sum_{j=1}^K p'_i(\sigma_j) = 1$. However, the algorithm from [Lemma 4.3](#) does not use this assumption. $\square$

Lemma 4.6. Let (X, Y) be an instance of the α -LCWS2 problem. In $O(n^5)$ time and space one can find a string s which is an $(\alpha, d - 1)$ -subsequence of both X and Y such that no $(\alpha, d + 1)$ -subsequence of both X and Y exists.

Proof. We set $\epsilon = 1/n$ and use the algorithm from Lemma 4.5 for the instance (X, Y, α, ϵ) . Thus in $O(n^5)$ time and space we obtain a string $z \in \Sigma^d$ which is an $(\alpha^{1+1/n}, d)$ -subsequence of X and an (α, d) -subsequence of Y . Note that no $(\alpha, d + 1)$ -subsequence of both X and Y exists. Now we remove exactly one character of the string z which has the smallest value of $p_{i_k}^{(X)}(z_k)$. Thus we obtain the final string $s \in \Sigma^{d-1}$ which is an $(\alpha, d - 1)$ -subsequence of both X and Y , since:

$$\alpha^{(1+1/n)(1-1/d)} \geq \alpha^{(1+1/n)(1-1/n)} = \alpha^{1-1/n^2} \geq \alpha. \quad \square$$

Theorem 4.7. For any real value $\epsilon \in (0, 1]$ there exists a $(1 - \epsilon)$ -approximation algorithm for the (α_1, α_2) -LCWS2 problem which runs in polynomial time and uses $O(n^5)$ space. Consequently the (α_1, α_2) -LCWS2 problem admits a PTAS.

Proof. Let (X', Y') be an instance of the (α_1, α_2) -LCWS2 problem. We perform the reduction from Lemma 2.9 to obtain an equivalent instance (X, Y) of the α -LCWS2 problem with just one parameter.

Using the algorithm from Lemma 4.6 we find a positive integer d and an $(\alpha, d - 1)$ -subsequence. If $d \geq 1/\epsilon$ then we are done since in that case we have $(d - 1)/d = 1 - 1/d \geq 1 - \epsilon$ which means that we have found a $(1 - \epsilon)$ -approximation.

If $d < 1/\epsilon$ then we search for an (α, d) -subsequence using a brute-force approach, i.e., we try all $\binom{|X|}{d}, \binom{|Y|}{d}$ subsets of positions in each sequence and all $|\Sigma|^d$ possible strings of length d , which leads to $n^{O(1/\epsilon)}$ running time. If we find an (α, d) -subsequence we simply return it as the final answer. Otherwise we return the $(\alpha, d - 1)$ -subsequence found by the algorithm from Lemma 4.6. Either way when $d < 1/\epsilon$ our answer is optimal. $\square$

5. Conclusions

The main problem considered in this paper is (α_1, α_2) -LCWS2, which is a natural extension of the longest common subsequence problem for biological weighted sequences. We have shown that this problem is NP-hard even for binary alphabet. Afterwards we have presented a $(1/2)$ -approximation algorithm working in $O(n^5)$ time and $O(n^2)$ space and a polynomial-time approximation scheme (PTAS) working in $O(n^5)$ space. Note that obtaining a fully polynomial-time approximation scheme (FPTAS) for the LCWS2 problem is not possible, since this would imply tractability of LCWS2.

Acknowledgements

We thank M. Christodoulakis, M. Crochemore, C. Iliopoulos and G. Tischler for helpful discussions about the weighted LCS problem.

References

- [1] A. Amir, E. Chencinski, C.S. Iliopoulos, T. Kopelowitz, H. Zhang, Property matching and weighted matching, Theoret. Comput. Sci. 395 (2-3) (2008) 298–310.
- [2] A. Amir, Z. Gotthilf, B.R. Shalom, Weighted LCS, J. Discrete Algorithms 8 (2010) 273–281.
- [3] A. Amir, C.S. Iliopoulos, O. Kapah, E. Porat, Approximate matching in weighted sequences, in: M. Lewenstein, G. Valiente (Eds.), CPM, in: Lecture Notes in Computer Science, vol. 4009, Springer, 2006, pp. 365–376.
- [4] P. Antoniou, C.S. Iliopoulos, L. Mouchard, S.P. Pissis, Algorithms for mapping short degenerate and weighted sequences to a reference genome, Int. J. Comput. Biol. Drug Des. 2 (4) (2009) 385–397.
- [5] C. Barton, S.P. Pissis, Optimal computation of all repetitions in a weighted string, in: C.S. Iliopoulos, A. Langiu (Eds.), ICABD, in: CEUR Workshop Proceedings, vol. 1146, CEUR-WS.org, 2014, pp. 9–15.
- [6] L. Bergroth, H. Hakonen, T. Raita, A survey of longest common subsequence algorithms, in: SPIRE, 2000, pp. 39–48.
- [7] M. Christodoulakis, C.S. Iliopoulos, L. Mouchard, K. Perdikuri, A.K. Tsakalidis, K. Tsiachas, Computation of repetitions and regularities of biologically weighted sequences, J. Comput. Biol. 13 (6) (2006) 1214–1231.
- [8] M. Cygan, M. Kubica, J. Radoszewski, W. Rytter, T. Waleń, Polynomial-time approximation algorithms for weighted LCS problem, in: R. Giancarlo, G. Manzini (Eds.), CPM, in: Lecture Notes in Computer Science, vol. 6661, Springer, 2011, pp. 455–466.
- [9] M.R. Garey, D.S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman, 1979.
- [10] D. Gusfield, Algorithms on Strings, Trees, and Sequences—Computer Science and Computational Biology, Cambridge University Press, 1997.
- [11] C.S. Iliopoulos, C. Makris, Y. Panagis, K. Perdikuri, E. Theodoridis, A.K. Tsakalidis, The weighted suffix tree: an efficient data structure for handling molecular weighted sequences and its applications, Fundam. Inform. 71 (2-3) (2006) 259–277.
- [12] C.S. Iliopoulos, M. Miller, S.P. Pissis, Parallel algorithms for degenerate and weighted sequences derived from high throughput sequencing technologies, in: J. Holub, J. Zdárek (Eds.), Stringology, Prague Stringology Club, Department of Computer Science and Engineering, Faculty of Electrical Engineering, Czech Technical University in Prague, 2009, pp. 249–262.
- [13] C.S. Iliopoulos, L. Mouchard, K. Perdikuri, A.K. Tsakalidis, Computing the repetitions in a biological weighted sequence, J. Autom. Lang. Comb. 10 (5/6) (2005) 687–696.
- [14] C.S. Iliopoulos, K. Perdikuri, E. Theodoridis, A.K. Tsakalidis, K. Tsiachas, Motif extraction from weighted sequences, in: A. Apostolico, M. Melucci (Eds.), SPIRE, in: Lecture Notes in Computer Science, vol. 3246, Springer, 2004, pp. 286–297.
- [15] E.W. Myers, Celera Genomics Corporation, A whole-genome assembly of drosophila, Science 287 (5461) (2000) 2196–2204.
- [16] J.D. Thompson, D.G. Higgins, T.J. Gibson, CLUSTAL W: improving the sensitivity of progressive multiple sequence alignment through sequence weighting, positions-specific gap penalties and weight matrix choice, Nucleic Acids Res. 22 (1994) 4673–4680.
- [17] J.C. Venter, Celera Genomics Corporation, The sequence of the human genome, Science 291 (2001) 1304–1351.
- [18] H. Zhang, Q. Guo, J. Fan, C.S. Iliopoulos, Loose and strict repeats in weighted sequences of proteins, Protein Peptide Lett. 17 (9) (2010) 1136–1142. (7).

- [19] H. Zhang, Q. Guo, C.S. Iliopoulos, String matching with swaps in a weighted sequence, in: J. Zhang, J.-H. He, Y. Fu (Eds.), CIS, in: *Lecture Notes in Computer Science*, vol. 3314, Springer, 2004, pp. 698–704.
- [20] H. Zhang, Q. Guo, C.S. Iliopoulos, An algorithmic framework for motif discovery problems in weighted sequences, in: T. Calamoneri, J. Díaz (Eds.), CIAC, in: *Lecture Notes in Computer Science*, vol. 6078, Springer, 2010, pp. 335–346.
- [21] H. Zhang, Q. Guo, C.S. Iliopoulos, Varieties of regularities in weighted sequences, in: B. Chen (Ed.), AAIM, in: *Lecture Notes in Computer Science*, vol. 6124, Springer, 2010, pp. 271–280.
- [22] H. Zhang, Q. Guo, C.S. Iliopoulos, Locating tandem repeats in weighted sequences in proteins, *BMC Bioinformatics* 14 (S-8) (2013) S2.